

Prof. Geraldo Braz Junior

Interface Gráfica

- A interface gráfica é composta de:
 - Componentes
 - Recipientes (Containers)
 - Layout

Principais Containers **Swing**

- JFrame (JInternalFrame)
- JPanel (JSplitPane, JRootPane, JTabbedPane, ...)
- JWindow
- Principais Métodos:
 - add(Component comp) ou add(Component comp, int index) ● Adiciona um componente ao container
 - remove(Component comp) ou remove(int index) ● Remove um componente do container

- removeAll()
 - Remove todos os componentes do container
- setLayout(LayoutManager mgr)
 - Configura o layout do container

Primeira Janela

```
import javax.swing.JFrame;
```

```
public class Moldura {  
    public static void main(String[] args) {  
        JFrame janela = new JFrame("Janela Principal"); //Aparece no título  
        janela.setSize(600, 400); //tamanho  
        janela.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
        janela.setVisible(true); //mostra o frame.  
    }  
}
```

JFrame.setDefaultCloseOperation()

- EXIT_ON_CLOSE
 - Encerra o programa quando clicar no X
 - `System.exit(0);`
- DISPOSE_ON_CLOSE
 - Destrói o frame quando clicar no X
- DO_NOTHING_ON_CLOSE
 - Não faz nada quando clicar no X
- HIDE_ON_CLOSE
 - Esconde o frame quando clicar no X
 - `JFrame.setVisible(false);`

Adicionando componentes ao JFrame

- ...

```
JPanel painel = new JPanel();  
painel.add(new JButton("Botão"));  
janela.add(painel);  
//para remover:  
janela.remove(painel);
```

- ...

Tratamento de eventos

- Interfaces gráficas são guiadas por eventos, por exemplo: ●

Movimento do mouse,

- Pressionar um botão,
- Entrada de dados em um textfield,
- Escolher um item em uma lista..

- Em Java eventos são objetos que descrevem o que aconteceu

Tratamento de eventos

- O modelo de eventos se torna fácil de usar, pois é baseado em uma nomenclatura simples:
 - Para cada tipo de evento ____Event existe uma interface ____Listener (interface “ouvinte”, que espera o evento acontecer)

- Cada interface possui um método que deve ser implementado e que será executado caso determinado evento aconteça (por exemplo: actionPerformed())
- ActionEvent: ActionListener
- ItemEvent : ItemListener
- TextEvent : TextListener
- WindowEvent: WindowListener
- KeyEvent : KeyListener

Tratamento de eventos

- Os métodos das interfaces são todos public void e recebem um único argumento do tipo ____Event
- Se um componente emite eventos de tipo ____Event, então ele tem um método público chamado add____Listener

- Quando um componente é ativado de uma maneira que o leve a emitir eventos de um tipo, todos os receptores registrados recebem uma chamada para um dos métodos da interface

Exemplo:

- Adicionamos um botão na interface:
- ...
`JButton confirmar = new JButton("Confirmar");`
`this.add(confirmar);`
- ...
- Para que este botão faça alguma coisa, precisamos associar ele a uma classe que implemente o ouvinte de eventos de um botão:

- O **JButton** possui o método **addActionListener**, então a interface necessária é a **ActionListener**
- E o evento gerado será o **ActionEvent**

Exemplo:

- Precisamos criar uma classe que implemente a interface `ActionListener`:

```
import java.awt.event.ActionEvent;  
import java.awt.event.ActionListener;
```

```
public class OuvinteBotao implements ActionListener{  
    // devemos sobrescrever o método desta interface:  
    @Override  
    public void actionPerformed(ActionEvent evento) {  
        ● //colocamos aqui o que deve ser feito quando o botão for clicado  
    }  
}
```

}

Exemplo:

- Por fim, registramos a classe ouvinte criada, como um dos ouvintes do botão:
- ...
 JButton confirmar = new JButton("Confirmar");
 confirmar.addActionListener(**new OuvinteBotao()**);
 this.add(confirmar);
- ...

Alguns eventos utilizados:

- **ActionEvent**
 - fonte: componentes de ação (JButton, JTextField, ...)
- **MouseEvent**
 - fonte: componentes afetados pelo mouse
- **ItemEvent**
 - fonte: componentes de seleção (JCheckBox, JRadioButton, JComboBox)
- **AdjustmentEvent**: fonte: scrollbars
- **TextEvent**: fonte: componentes de texto
- **WindowEvent**: fonte: janelas

- FocusEvent: fonte: componentes em geral
- KeyEvent: fonte: componentes afetados pelo teclado

Alguns componentes:

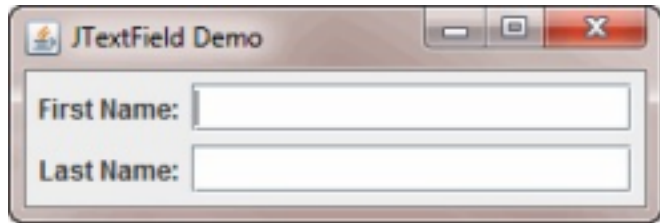
- JLabel:
 - São rótulos usadas para identificar outros elementos. Normalmente são inseridas a esquerda ou acima do elemento que esta sendo rotulado
 - Pode apresentar: Somente Texto, Imagens, Texto e imagens
 - Olhe o exemplo Teste_Label



Alguns componentes

- **JTextField** e **JPasswordField**:
 - Dispositivo de linha única de texto, em virtude disso, um `ActionListener` pode ser informado – através de `actionPerformed()`
 - quando a tecla enter for pressionada.
 - Este componente não exibe barra de rolagem, mas é possível rolar sobre o texto longo da esquerda para a direita, se for necessário.
 - A classe **JPasswordField** herda de **JTextField** e adiciona vários métodos que são específicos ao processamento de senhas.

- Olhe o exemplo Teste_TextField.



Alguns componentes

- Botões:

- São elementos de forma retangular que o usuário utiliza para iniciar ações.

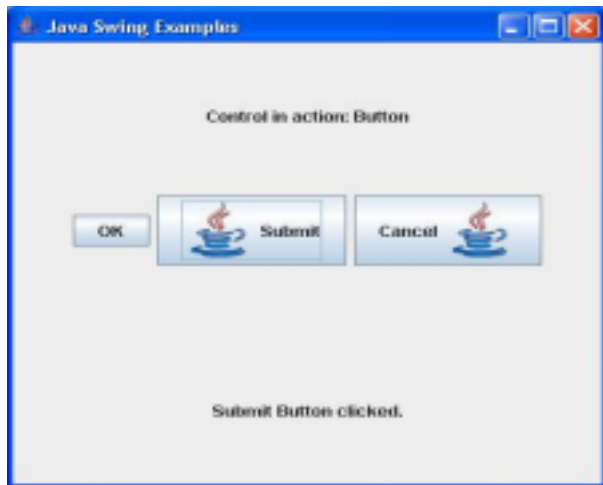
- O pacote swing define vários tipos de botões: botões de comando, **Checkbox** (caixa de seleção), **Radio buttons**,...

- Todos estes botões são subclasse de `javax.swing.AbstractButton`.

Alguns componentes

- **JButton:**

- Quando o usuário clica no botão é gerado um `ActionEvent`, o método `actionPerformed(ActionEvent evt)` de qualquer “ouvinte” registrado é chamado quando o botão for pressionado.



Alguns componentes

● **JCheckBox:**

- Fornece um dispositivo simples de entrada “liga/desliga” com um rótulo de texto ao lado
- A seleção ou o cancelamento de um checkbox é notificado pela interface `ItemListener` que define o método: `public void itemStateChanged(ItemEvent e)`
- Um objeto da classe `ItemEvent` possui um método `getStateChange()`, que retorna uma das duas constantes:

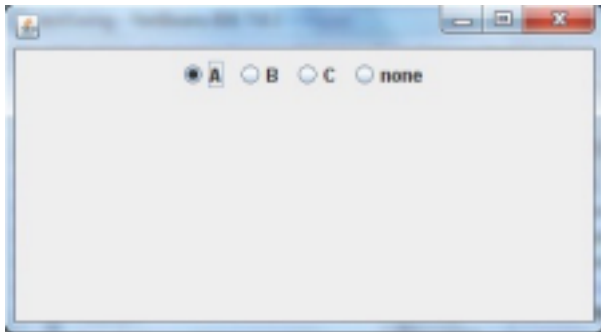
- ItemEvent.SELECTED
- ItemEvent.DESELECTED



Alguns componentes

● **JRadioButton** e **ButtonGroup**:

- Os botões de opção normalmente aparecem como um grupo em que apenas um botão de opção pode ser selecionado.
- O relacionamento lógico entre botões de um grupo é mantido por um objeto ButtonGroup.
- O objeto ButtonGroup em si não é um componente GUI, pois não é exibido em uma interface com o usuário, o mesmo pode ser utilizado como parâmetro no construtor de um JRadioButton.



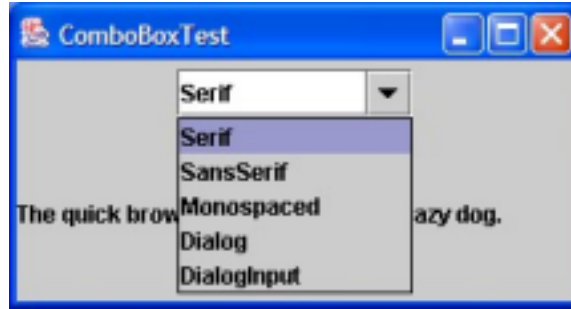
Alguns componentes

- **JToolTip** (Dica):

- É uma string de texto que fornece uma dica.
- Aparece automaticamente quando o mouse permanece sobre o componente.
- Pode ser configurado para qualquer JComponent.
- É necessário apenas chamar o método `setToolTipText()` no componente. ● Ex: `comp.setToolTipText("dica");`

Alguns componentes

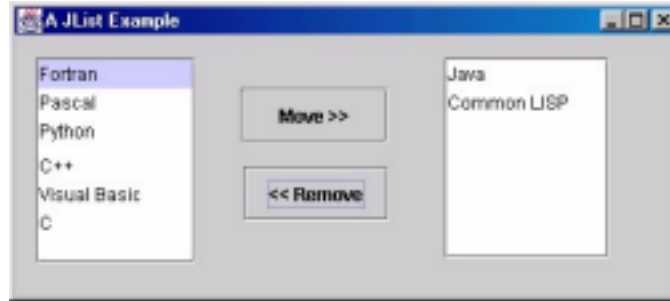
- **JComboBox** (Escolhas):
 - Fornece uma lista de itens entre os quais o usuário pode escolher.
 - JComboBox gera ItemEvent, assim como, JCheckBox e JRadioButton.



Alguns componentes

- **JList:**

- Exibe uma série de itens em que o usuário pode selecionar um ou mais itens.
 - Este componente suporta listas de uma única seleção (exemplo Teste_List) ou lista de múltipla seleção (exemplo Teste_MultipleSelection).



Tratando eventos do mouse

- Os eventos do mouse podem ser capturados por qualquer componente
- O tratamento funciona igual a qualquer evento:
 - Deve-se criar uma classe ouvinte que implemente a interface `MouseListener` ou `MouseMotionListener`
 - Depois, deve-se registrar no componente a classe

- ouvinte: ● `Componente.addMouseListener` ou
● `Componente.addMouseMotionListener`

Tratando eventos do mouse

- Métodos da interface `MouseListener`:
 - **`mouseClicked`**: Quando o botão do mouse for pressionado e liberado.
 - **`mousePressed`**: Quando o botão do mouse for pressionado.
 - **`mouseReleased`**: Quando o botão do mouse for liberado (precedido por um evento `mousePressed`).
 - **`mouseEntered`**: Quando o cursor do mouse entra nos limites de um componente.
 - **`mouseExited`**: Quando o cursor do mouse deixa os limites de um componente.
 - **`mouseDragged`**: Quando o botão do mouse for pressionado e movido (precedido por uma chamada `mousePressed`).

- **mouseMoved**: Quando o cursor do mouse for movido nos limites de um componente.

Tratamento eventos do teclado

- Uma classe que implementa `KeyListener` deve fornecer as definições para os métodos: `KeyPressed`, `KeyReleased` e `KeyTyped`.

Classes Adaptadoras

- Algumas interfaces possuem uma classe adaptadora que implementa todos os seus métodos, com um corpo vazio.
- Só existem para interfaces que possuem mais de um método, pode-se herdar de uma classe adaptadora e fornecer uma implementação (sobrescrever) somente o método necessário para o tratamento do evento.

- O nome da classe adaptadora é semelhante ao do Listener:
 - MouseListener: MouseAdapter
 - MouseMotionListener: MouseMotionAdapter
 - KeyListener: KeyAdapter
 - WindowListener: WindowAdapter

Exercício:

- Crie um programa em Java, utilizando a biblioteca Swing para elaborar a interface gráfica, semelhante à imagem abaixo.
- O programa deve possibilitar a entrada de dois valores inteiros e deve somar estes valores colocando o resultado em um terceiro campo. As exceções devem ser tratadas.
- Mostre uma dica se o mouse permanecer sobre o botão =

